

Image Classification of Crop Diseases Using Deep Learning

A Comparative Study of Custom CNNs and Transfer Learning

Ethan Hazari
Software Engineering Technology
McMaster University
Hamilton, Canada
1280 Main St W

Abstract

Object recognition and image classification are typically ways to make machines react after receiving signals from a camera. The purpose of this project is to accurately detect crop disease from leaf images to help improve agricultural yield and food security. Using Deep learning algorithms to train models with pre-collected image data enables the machine to understand what it is detecting. This report compares a custom-built Convolutional Neural Network (CNN) against pre-trained ResNet50 model to evaluate the trade-offs between computational time and classification precisions.

Introduction

In the future, automated agricultural monitoring will become increasingly vital to sustain global food supplies by implementing machine learning identifying crop diseases becomes faster and more scalable than relying on human expertise. Machine learning uses given data to tell the computer what we are looking for and what to do next when it identifies the target object. Because there are many repetitive tasks in agriculture, using a machine to replace human visual inspection can make finishing the task easier. Different algorithms will affect the speed of reaction and the accuracy of the result. By understanding the basic concept of machine learning for image classification, we must break the process into three main parts: collection data, making the model, and testing the result. In this project, we are going to use a custom simple CNN and an open-source ResNet50 model from the PyTorch library to detect healthy and diseased crop leaves.

Methodology

A. Data Collection and Preparation

To train the model, data needs to be sent to program to let it know which objects it needs to learn. First, a dataset consisting of thousands of images of crop leaves was utilized. In this case, five different categories of objects are classified: four specific disease states and one healthy state.

B. Data Training

For the training data, two different architectures were used to form a baseline and an advanced comparison. First, a Simple CNN was built from scratch. Second, the ResNet50 model, utilizing transfer learning, was imported.

The final fully connected layer of the ResNet50 model was modified to output exactly 5 classes. To ensure reliable testing without data bias, a 3-Fold Cross-Validation method was implemented, utilizing a batch size of 32 and learning rate of 0.001 across 5 epochs per fold.





Image Preprocessing and Normalization Pipeline

To ensure the neural network processes the images uniformly, a data transformation pipeline was established. This pipeline guarantees all images are in RGB color format, resizes them to a standard 224x224 pixel resolution, converts them into PyTorch tensors, and applies statistical normalization. This scaling helps the neural network learn features much faster and prevents unstable weight updates during training.

```
# Custom class to enforce 3 color channels (RGB)
class ConvertToRGB(object):
    def __call__(self, img):
        if img.mode != "RGB":
            img = img.convert("RGB")
        return img

# The transformation pipeline
transform_normalized = transforms.Compose([
    ConvertToRGB(),
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])
```

Custom Baseline CNN Architecture

The baseline model, SimpleCNN, was built from scratch using PyTorch. It consists of two convolutional layers to extract spatial features (such as edges and textures) using 3x3 sliding filters. Max pooling layers are used to compress the image size while retaining the most important features. Finally, the multi-dimensional feature maps are flattened and passed through fully connected linear layers to output predictions for the 5 specific crop classes.

```
class SimpleCNN(nn.Module):
    def __init__(self, num_classes=5):
        super(SimpleCNN, self).__init__()
        # 1st Convolutional Layer & Max Pooling
        self.conv1 = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

        # 2nd Convolutional Layer
        self.conv2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=3, padding=1)

        # Fully Connected Layers to classify into the 5 categories
        self.fc1 = nn.Linear(32 * 56 * 56, 128)
        self.fc2 = nn.Linear(128, num_classes)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 32 * 56 * 56) # Flatten for linear layers
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```



Overcoming I/O Bottlenecks with Multi-Threading

To drastically reduce total computation time, the dataset loading process was optimized using multi-threading. By configuring the PyTorch DataLoader with num_workers=2 and pin_memory=True, the system uses background CPU threads to fetch and prepare upcoming image batches while the GPU actively trains on the current batch. This eliminated severe Input/Output (I/O) bottlenecks.

```
# Optimized DataLoaders for rapid GPU training
train_loader = DataLoader(
    dataset,
    batch_size=batch_size,
    sampler=train_subsampler,
    num_workers=2,          # Background thread fetching
    pin_memory=True        # Fast-lane memory transfer to GPU
)
```

Robust Testing via K-Fold Cross Validation

To ensure the model's accuracy was not reliant on a "lucky" data split, a strict 3-Fold Cross-Validation mechanism was implemented using Scikit-Learn. The dataset is shuffled to prevent bias and divided into three distinct segments. The model rotates through these segments, training on two parts and validating on the third, resulting in a highly reliable average accuracy metric.

```
from sklearn.model_selection import KFold
from torch.utils.data import SubsetRandomSampler

# Initialize the K-Fold splitter
kfold = KFold(n_splits=k_folds, shuffle=True, random_state=42)

# Loop through each fold to split the training and validation data
for fold, (train_ids, val_ids) in enumerate(kfold.split(dataset)):
    # Create random samplers for the specific fold indices
    train_subsampler = SubsetRandomSampler(train_ids)
    val_subsampler = SubsetRandomSampler(val_ids)

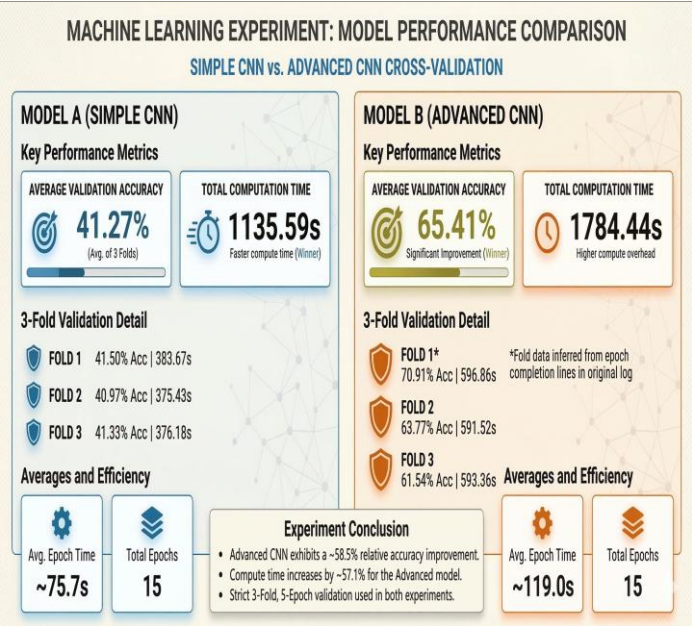
    # ... [Model Initialization and Epoch Loop runs here] ...
```

Test Result

It is good to test results after training to understand the real-world viability of the models. Based on the data shown, the 3-Fold Cross-Validation for the Simple CNN yielded an average accuracy of **41.27%** with a total computation time of **1135.59** seconds.

In a real-life situation where precision is paramount, 41% accuracy is low. However, considering there are 5 classes where random guessing yields a 20% accuracy, this result is acceptable for a lightweight, baseline model trained from scratch over only 5 epochs.

Conversely, the advanced ResNet50 model, utilizing transfer learning, yielded a significantly improved average validation accuracy of **65.41%** peaking at **70.91%** during the first fold. This represents an approximated **58.5%** relative improvement in classification precision over baseline Simple CNN. However, this increase in performance came at the direct cost of higher computational overhead. The total computation time for the RestNet50 model was **1784.44** seconds, marking a **57.1%** increase in training time compared to the baseline model. Ultimately, this shows a clear operational trade-off, while the advanced, pre-trained architecture requires greater computational resources and time, it delivers a substantial and necessary boost in accuracy that brings the system closer to real-world agricultural viability.



References

PyTorch Contributors. "Models and pre-trained weights — Torchvision 0.16 documentation." PyTorch.org. <https://pytorch.org/vision/stable/models.html> (accessed Mar. 10, 2026).

K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 2016, pp. 770-778.

WorldQuant University, "Crop Disease Image Dataset," Applied Data Science / Machine Learning Course Materials. [Online dataset]. Available via WorldQuant University learning portal.

WorldQuant University, "Deep Learning and Convolutional Neural Networks," Course Lectures and Programming Materials. [Online]. Available: <https://www.wqu.edu/>